

Get a Unique List of Objects in an Array of Object in JavaScript



Last Updated on 2022-07-11 by [Yagi](#)

Recently, I needed a way to ensure that my JavaScript array of objects does not contain any duplicate objects based on an 'id' key. While I do enjoy more common approaches to this solution with the use of a simple 'for' loop it is always a bit of fun to see how folks have come up with modern solutions.

It wasn't too long before I stumbled across this ES6 one-liner:

```
1 let uniqueObjArray = [...new Map(objArray.map((item) => [item["id"], item]))];
```

Pretty neat, huh?



Here are two locations I found them:

- [Stackoverflow – Savan Akbari](#)
- [Stackoverflow – Arun Saini](#)

But as many code oneliners are, they are a little tricky to understand. So I wanted to spend some time understanding how this worked and thought, you might find this a little interesting too.

Let's take a look at an example and then work through each bit as we go.



Table of Contents



1. [The Example](#)
2. [Code Breakdown](#)
 - 2.1. [myObjArray.map](#)
 - 2.2. [new Map](#)
 - 2.2.1. [A quick side example](#)
 - 2.2.2. [new Map of our example](#)
 - 2.3. [Displaying the keys of each property in the Map](#)
 - 2.4. [Displaying the values of each property in the Map](#)



- [2.5. Using the spread syntax to create our array of object](#)
- [3. Performance](#)
- [4. Conclusion](#)



Sorry, you don't have permission to

Website blocked by TCS policy

Not allowed to browse **Video Streaming** category

You tried to visit: <https://www.youtube.com/embed/10Celversion=3&rel=1&showsearch=0&showinf...>

Need help? Contact our support team at 04446324,04278848,65, [InternalIS.SecO](#)



```
1  const myObjArray = [  
2    {  
3      name: "Eva Devore",  
4      character: "Evandra",  
5      episodes: 15,  
6    },  
7    {  
8      name: "Alessia Medina",  
9      character: "Nixie",  
10     episodes: 15,  
11   },  
12   {  
13     name: "Kendall Drury",  
14     character: "DM",  
15     episodes: 15,  
16   },  
17   {  
18     name: "Thomas Taufan",  
19     character: "Antrius",  
20     episodes: 14,  
21   },  
22 ]
```



```
22 {
23   name: "Alessia Medina",
24   character: "Nixie",
25   episodes: 15,
26 },
27 ];
28
29 // Creates an array of objects with unique "name" property values.
30 let uniqueObjArray = [
31   ...new Map(myObjArray.map((item) => [item["name"], item])).values()
32 ];
33
34 console.log("uniqueObjArray", uniqueObjArray);
35
36 // LOGS:
37 // uniqueObjArray [
38 //   { name: 'Eva Devore', character: 'Evandra', episodes: 15 },
39 //   { name: 'Alessia Medina', character: 'Nixie', episodes: 15 },
40 //   { name: 'Kendall Drury', character: 'DM', episodes: 15 },
41 //   { name: 'Thomas Taufan', character: 'Antrius', episodes: 14 }
42 // ]
```

In the example above, we want to create a unique array of objects based on the 'name' property of each object set.

You can see that there is a duplicate property in positions 1 and 4 with key 'name' and value, 'Alessia Medina' that we need to remove.

You can also change the key to either the 'character' or 'episodes'

property



essentially reassign the 'name' property each time it loops through the array when creating the new map.

निजता आपके नियंत्रण में है

आसान चरण में FB प्रोफाइल लॉक करें
और नियंत्रित करें कि साझा सामग्री व
देख सकता है



Let's start breaking down the script so we can see how each one operates.

Code Breakdown

myObjArray.map

```
1 // Creates a new 2d array with the selected key in position 0 and the ob
2 let test_uniqueObjArray_map = myObjArray.map((item) => {
3   return [item["name"], item];
4 });
5
6 // LOGS: test_uniqueObjArray_map
7 // [
8 //   ["Eva Devore", { name: "Eva Devore", character: "Evandra", episod
9 //   ["Alessia Medina",{ name: "Alessia Medina", character: "Nixie", e
10 //   ["Kendall Drury", { name: "Kendall Drury", character: "DM", episod
11 //   ["Thomas Taufan", { name: "Thomas Taufan", character: "Antrius",
12 //   ["Alessia Medina",{ name: "Alessia Medina", character: "Nixie", e
13 // ];
```

The first task of this script remaps the array of objects using the JavaScript [map\(\)](#) method. This method takes a function, which in our case is an [arrow function](#).



```
1 myArray.map(element) => <em>return new element as a result of a callback</em>
```

As an ordinary function, it would look like this:

```
1 myArray.map(function(element){
2   return <em>new element as a result of a callback</em>
3 })
```

In our example above, we have our callback arguments on a new line so we will also need to include curly braces `{}`.

With the map method, the function will act on each array and return the result to generate a new array of the same length.



For us, our call back condition rebuilds each array to make a sub-array containing the value of each name key in the array as the zeroeth element and the object at the first element.

```
1 ...  
2  
3 [item["name"], item];  
4  
5 ...
```

So the first element in the new array will look like this:

```
1 ...  
2  
3 ["Eva Devore", { name: "Eva Devore", character: "Evandra", episodes: 15 }]  
4  
5 ...
```

new Map

A QUICK SIDE EXAMPLE

Before we continue, let's take a quick look at a basic Map process on



```
3 ["key_one", "val_one"],  
4 ["key_two", "val_two"],  
5 ["key_three", "val_three"],  
6 ];  
7 console.log("valuesObject", valuesObject);  
8  
9 // LOGS: valuesObject Map {  
10 //   'key_one' => 'val_one',  
11 //   'key_two' => 'val_two',  
12 //   'key_three' => 'val_three'  
13 // }
```

To be frank, I didn't really understand the Map object too well until I explored this script.



[Map](#) object stores key-value pairs similar to an Object. However, the Map maintains the insertion order of the properties. You'll see Map objects often displayed like this when logged out in the console.

```
1 'key' => 'value'
```

Map can be iterated through in a similar way to a 2d array with the zeroeth element as a key and the next element as a value for each property of the map – `['key', 'value']`.

[Back to our main example...](#)

NEW MAP OF OUR EXAMPLE

```
1 // PREVIOUS LOG: test_uniqueObjArray_map
2 // [
3 //   ["Eva Devore", { name: "Eva Devore", character: "Evandra", episode: 1,
4 //   ["Alessia Medina", { name: "Alessia Medina", character: "Nixie", episode: 2,
5 //   ["Kendall Drury", { name: "Kendall Drury", character: "DM", episode: 3,
6 //   ["Thomas Taufan", { name: "Thomas Taufan", character: "Antrius", episode: 4,
7 //   ["Alessia Medina", { name: "Alessia Medina", character: "Nixie", episode: 5,
8 // ];
9
10 // Create a new Map from our mapped data.
11 let test_uniqueObjArray_NewMap = new Map(test_uniqueObjArray_map);
12
13 // LOGS: test_uniqueObjArray_NewMap Map {
14 //   'Eva Devore' => { name: 'Eva Devore', character: 'Evandra', episode: 1,
```



```
15 // 'Alessia Medina' => { name: 'Alessia Medina', character: 'Nixie', episode: '1' }  
16 // 'Kendall Drury' => { name: 'Kendall Drury', character: 'DM', episode: '2' }  
17 // 'Thomas Taufan' => { name: 'Thomas Taufan', character: 'Antrius', episode: '3' }  
18 // }
```

We are using the data we retrieved from our previous example here to remove some of the clutter from the process. I have added those results at the top of the code block above.

In this example, we simply apply `new Map` to this array of data. By doing this Map turns into a type of Object with a key-value pair. Now keep in mind that Object keys are the highlander of data types – there can be only one.



What does this mean beyond a bad joke that really shows my age?

It means that each key must be unique. All of our keys are now the names of our users. The new Map constructor process will then iterate through each name and store it and then assign its value. If a key already exists it will overwrite it with this next value with the same key name.

This means that the last duplicate key will always be displayed. Effectively only storing unique values.



Displaying the keys of each property in the Map

We can generate [iterators](#) to go through each key or value with the `keys()` and `values()` methods respectively.

We will have a look at the [keys\(\)](#) method first quickly.

Let's apply `keys()` to our `test_uniqueObjArray_NewMap Map` we generated above.

```
1 // PREVIOUS LOG: test_uniqueObjArray_NewMap Map {  
  
8 console.log("test_uniqueObjArray_NewMap_keys", test_uniqueObjArray_NewMap_keys);  
9  
10 // LOGS: test_uniqueObjArray_NewMap_keys [Map Iterator] {  
11 //   'Eva Devore',  
12 //   'Alessia Medina',  
13 //   'Kendall Drury',  
14 //   'Thomas Taufan'  
15 // }
```

As you can see this produces an iterator of all the (unique) keys in our data as a Map Iterator. It's not quite an array of objects, but it allows us to iterate over each key to do something with it.

The same is true for the `values()` method.



Displaying the values of each property in the Map

```
1 // PREVIOUS LOG: test_uniqueObjArray_NewMap Map {
2 //   'Eva Devore' => { name: 'Eva Devore', character: 'Evandra', episodes: 15 },
3 //   'Alessia Medina' => { name: 'Alessia Medina', character: 'Nixie', episodes: 15 },
4 //   'Kendall Drury' => { name: 'Kendall Drury', character: 'DM', episodes: 15 },
5 //   'Thomas Taufan' => { name: 'Thomas Taufan', character: 'Antrius', episodes: 14 }
6 // }
7
8 let test_uniqueObjArray_NewMap_values = test_uniqueObjArray_NewMap.values()
9
10 console.log(
11   "test_uniqueObjArray_NewMap_values",
12   test_uniqueObjArray_NewMap_values
13 );
14
15
16 // LOGS: test_uniqueObjArray_NewMap_values [Map Iterator] {
17 //   { name: 'Eva Devore', character: 'Evandra', episodes: 15 },
18 //   { name: 'Alessia Medina', character: 'Nixie', episodes: 15 },
19 //   { name: 'Kendall Drury', character: 'DM', episodes: 15 },
20 //   { name: 'Thomas Taufan', character: 'Antrius', episodes: 14 }
21 // }
```

Here we want to get an iterator of our values so that we can recreate an array of objects again.

Using the [values\(\)](#) iterator method we now have our Map values ready to go.



Now that we have an iterator of our unique values we can now place them in our [spread syntax](#) – “`...`”.

When you apply the spread syntax on an array, it will add each item of an iterable to the array. Take a look at what it does to our Map values.

```
1 // PREVIOUS LOG: test_uniqueObjArray_NewMap_values [Map Iterator] {
2 //   { name: 'Eva Devore', character: 'Evandra', episodes: 15 },
3 //   { name: 'Alessia Medina', character: 'Nixie', episodes: 15 },
4 //   { name: 'Kendall Drury', character: 'DM', episodes: 15 },
5 //   { name: 'Thomas Taufan', character: 'Antrius', episodes: 14 }
6 // }
7
8 let test_uniqueObjArray_NewMap_values_asArray = [
9   ...test_uniqueObjArray_NewMap_values,
```



```
10 ];
11
12 // LOGS: test_uniqueObjArray_NewMap_values_asArray [
13 //   { name: 'Eva Devore', character: 'Evandra', episodes: 15 },
14 //   { name: 'Alessia Medina', character: 'Nixie', episodes: 15 },
15 //   { name: 'Kendall Drury', character: 'DM', episodes: 15 },
16 //   { name: 'Thomas Taufan', character: 'Antrius', episodes: 14 }
17 // ]
```

This is similar to using the `Array.from()` static method that would look like this:

```
1 let test_uniqueObjArray_NewMap_values_asArray = Array.from(
2   test_uniqueObjArray_NewMap_values
3 );
```

Performance

So how does this one-liner stack up against a more traditional for-loop like this?

```
1 var array = [
2   {
3     name: "Eva Devore",
4     character: "Evandra",
5     episodes: 15,
6   },
7   {
8     name: "Kendall Drury",
9     character: "DM",
10    episodes: 15,
11  },
12  {
13    name: "Thomas Taufan",
14    character: "Antrius",
15    episodes: 14,
16  },
17  {
18    name: "Alessia Medina",
19    character: "Nixie",
20    episodes: 15,
21  },
22 ];
23
24 var unique = [];
25 var distinct = [];
26 for( let i = 0; i < array.length; i++ ){
27   if( !unique[array[i].name]){
28     distinct.push(array[i].name);
29     unique[array[i].name] = 1;
30   }
31 }
```



```
34 }  
35 }  
36  
37 console.log(distinct)
```

Surprisingly better than I thought it would, to be honest.

Running a benchmark test with jsbench.me, the one-liner ran only 13.74% slower. Which is pretty good compared to some of the other options I found out there.

Conclusion

So should you be using this oneliner over the `for` loop? Is an impressive one-liner better than something more clear? To be honest, I am on the fence.

I do like the way this script operates. It is clean and once I got my head around the Map object, it did make a lot of sense. I think if I saw something like this in the wild I could pretty easily identify what it was for and see that it was a nice short solution to a problem.

if I need something in my toolkit to solve a problem like this, then I am definitely going to use it.

I have an example of how I used the code to check for duplicate selections of files in Google Drive here:

[Create a Google Workspace Add-on file picker card with CardService that opens a Google Picker in an overlay window – Google Apps Script](#)

